

SOW-BKI257 Knowledge Representation

Practical Assignment

Assignment: Consistency-based diagnosis

Model-based reasoning is one of the central topics of knowledge representation and reasoning in artificial intelligence. This assignment aims at testing your understanding of model-based reasoning and your ability to program standard algorithms by implementing in Scala the hitting set tree algorithm which you have seen during the lecture and which is also covered in the corresponding chapter of the lecture notes.

1 The Hitting Set Tree Algorithm

Optional warm-up exercises To get started, think first about a suitable way of representing (rooted and labelled) trees which facilitates straightforward performance of the kinds of operations which are part of the hitting set tree algorithm, including reading off the (minimal) hitting sets. To develop a feeling for how to construct and operate on these representations of trees, try doing the following:

- (i) Write a function which takes as input two integers b and d and outputs a rooted tree where every non-leaf node has exactly b children, every leaf node is at depth d , and every node is labelled by its depth (so that in particular the root r of the tree has label 0 and the leaves each have label d).
- (ii) Write a function which takes as input a tree and outputs the number of nodes in this tree. (Hint: Recall that trees created by the function from (i) should have exactly $\frac{b^{d+1}-1}{b-1}$ nodes. You can use this fact as part of verifying whether your functions behave as intended.)

1.1 Implementation of the hitting-set algorithm

Task 1: Generate conflict sets To get started, download `kr-practical.zip` from Brightspace and follow the instructions on how to use these files. Have a look at `Conflicts.scala` which contains the code for the theorem prover to be used to determine conflict sets, and at `Diagnosis.scala` which contains three small-scale diagnostic problems which you will use for testing your implementation of the hitting set tree algorithm. It also contains a slightly more elaborate problem in the form of a full adder, also covered in [1].

Experiment with using the theorem prover (tpf) in order to determine conflict sets for the given problems. For each problem, explicitly provide at least one conflict set which you found in this way. For one of these conflict sets, provide a proof that it is indeed a conflict set. You are not required to give a fully explicit logical derivation, rather an informal proof written in natural language will suffice, but in any case your argument should make clear reference to the relevant formal concepts (such as what a conflict set is).

Furthermore, in your final report you should provide a verbal description of how the theorem prover operates (what its inputs are, a high-level account of what it does with its input¹, and what its outputs are), and how your implementation uses the theorem prover (i.e. at which points is it called upon and to which purpose).

¹Unfortunately, it does *not* actually use the resolution rule, as this feature was lost during the translation from Prolog (which does natively use the resolution rule) to Scala. This discrepancy is called out to preemptively avoid any possible confusion.

Task 2: Implementation Write your own implementation of the hitting set algorithm. Observe that the algorithm can be subdivided into three separate components: constructing the hitting set tree for the diagnostic problem, reading off the hitting sets from this tree, and selecting from these the minimal hitting sets which are the diagnoses for the problem. This corresponds to writing the following three functions.

- First, `makeHittingTree`, which generates a hitting tree from a description of a diagnostic problem. Generally speaking, this can be achieved in two ways. The first option is to use the theorem prover to generate all the conflict sets in advance and implement the hitting tree algorithm as described on the slides from Lecture 3. Once you understand what this requires, you should be able to recognise this as an unnecessary duplication of work, hence *this option is not the recommended one*. The second, more efficient (and hence “correct”) option is to use the theorem prover on demand to find a new conflict set or determine that no further conflict set exists at the relevant steps during the construction of the hitting tree. Regardless of your approach, think in particular about the labels which you want to add to certain parts of the tree in order to keep track of the potential hitting set constructed thus far.
- Now that your program is able to generate hitting set trees, we require a function `gatherHittingSets` which can take such a tree as its input and read off the hitting sets which correspond to (paths from the root to) the leaves of the tree. Use a list to store and update your results.
- Finally, since a diagnosis for the diagnostic problem corresponds to a *minimal* hitting set for the conflict sets, you want a function `getDiagnoses` which takes a list of (hitting) sets as produced as by previous function and refines it so that only the minimal sets are left. This can be achieved by removing all supersets from the list – as the most straightforward task out of the three, this should pose the least difficulty, but if you struggle here you should be able to find plenty of examples online.

In the end, these three functions (which may freely call on the theorem prover) should be combined together into one main function, which should take a diagnostic problem as an input (thus again a System Description, the Components of the system and some Observations) and return a list of all its possible diagnoses.

Task 3: (not required) Evaluating the performance Optionally, add some of the optimisations to prune the search space as described in [1] (you should have access to the paper through your library account). Modify the example problems to become larger and more complex and investigate the time and space limits of your implementation of the hitting set tree algorithm. You can also compare the differences in performance between generating all conflicts sets up front versus generating one on demand. This part can add up to a 10% bonus on your grade; note that the final grade cannot exceed 100.

Task 4: Complete your report Write a report for your assignment. In your report you should evaluate your program by determining the minimal diagnoses for the given diagnostic problems. Explain why the obtained results are correct (or not correct). Also, reflect on your code, such as: What are the limitations? How can it be improved? What are the problems encountered? Do the (optional) optimisations help? What can you say about its complexity? The explicit evaluation criteria are provided in a separate document.

Acknowledgement

This assignment has been used in an alternative (Prolog-based) form for several years in the preceding course Representation and Interaction (and its earlier iterations known under yet other names). The author expresses his gratitude to Perry Groot and Bob de Ruiter for their work on these earlier versions, and to Maris Galesloot for his work on translating the assignment to Scala.

References

- [1] R. Reiter (1987). A theory of diagnosis from first principles. *Artificial Intelligence*, **32**, 57–95.